

Health Monitoring

The health module is module ID 4. It uses passive field watches to assess subsystem health without running a dedicated workload. The module is loaded when health watches or checks are first requested.

Use [dcgmi health](#) for passive health-watch syntax and [dcgmi diag](#) for active diagnostic suites, canonical test names, parameters, output, and exit status.

When health monitoring is useful

Health monitoring is the right starting point when applications must remain running and the question is whether recent telemetry contains evidence of a known problem. It can detect and summarize conditions involving PCIe, GPU memory, InfoROM, the driver, thermal and power behavior, NVLink, NVSwitch, NVIDIA Internode Memory Exchange (IMEX), and ConnectX devices, as well as supported NVIDIA CPU thermal and power conditions. Common uses include a node-wide background watch, a check at job completion, and a quick incident query after a workload reports an error.

The module interprets retained field samples; it does not apply load to prove that a GPU can execute a workload. A `Healthy` result means that no enabled health rule found an incident in the retained data. It does not mean that every subsystem was watched or that an active stress test passed.

Health, policy, and diagnostics occupy different points in a workflow:

- Core field watches collect the underlying measurements.
- Health converts selected measurements and events into warning or failure incidents that can be polled.
- Policy watches a smaller set of conditions and can notify a listener or request an action when a condition occurs.
- Diagnostics run active tests when the target hardware can be taken out of ordinary service.

Example: watch a scheduler allocation

Suppose GPUs 0 and 1 have been assigned to a job. Create a group and enable the passive GPU watches before the workload starts:

```
$ dcgmi group --create job-4815 --add gpu:0,gpu:1
$ dcgmi health --set pmitnd --group <group-id> \
  --update-interval 30 --max-keep-age 600
```

Replace `<group-id>` with the ID printed by the create command. The selector enables PCIe (`p`), memory (`m`), InfoROM (`i`), thermal and power (`t`), NVLink (`n`), and driver (`d`) watches. The setting remains in the running host engine after these commands exit.

After the watches have collected data, check the group during or after the job:

```
$ dcgmi health --fetch --group <group-id>
$ dcgmi health --check --group <group-id>
```

`--fetch` confirms what is enabled; it does not assess health. `--check` reports the overall severity and every affected entity and subsystem. Some watches require approximately 60 seconds of samples, so an immediate first check may not be meaningful.

During job cleanup, disable the watches and remove the transient group:

```
$ dcgmi health --clear --group <group-id>
$ dcgmi group --delete <group-id>
```

For a long-lived node monitor, use a stable group and leave the watches enabled instead. Choose a retention window long enough to preserve the event until the monitoring system performs its check.

Choosing passive or active checks

Use the available mechanisms according to the operational window:

- **Background health checks** are passive and can remain enabled while jobs run.
- **Prologue diagnostics** use the quick suite to test readiness before a job starts.
- **Epilogue diagnostics** use a medium suite or selected tests after a failed job, when the GPU is no longer serving the workload.
- **Full system validation** uses longer suites during administrator-led isolation and investigation.

Warning

All of these are online diagnostics, meaning they run within the current environment. There is potential for factors beyond the GPU to influence behavior in negative ways. While these tools try to identify those situations, full offline diagnostics delivered via a different NVIDIA tool are required for complete hardware validation, and are required for RMA.

Background Health Checks

This form of health check is based on passive, background monitoring of various hardware and software components. The objective is to identify problems in key areas with low monitoring overhead and without applying a diagnostic workload. These kinds of checks can catch serious problems, such as unresponsive GPUs, corrupted firmware, thermal escapes, etc.

When such issues are identified they are reported by DCGM as warnings or errors. Each situation can require different client responses, but the following guidelines are usually true:

- Warning - an issue has been detected that won't prevent current work from completing, but the issue should be examined and potentially addressed in the future.
- Failure - a critical issue has been detected and the current work is likely compromised or interrupted. These situations typically correspond to fatal RAS events and usually indicate the need for job termination and GPU health analysis.

Background health checks can be set and watched via simple DCGM interfaces. Using `dcgmi health` as the interface, the following code sets several health checks for a group and then verifies that those checks are currently enabled:

```
$ dcgmi health -g 1 -s mpi
```

```
Health monitor systems set successfully.
```

The available health watch flags are:

- `p` - PCIe watches
- `m` - Memory watches
- `i` - InfoROM watches
- `t` - Thermal and power watches
- `n` - NVLink watches
- `d` - Driver watches
- `e` - NVIDIA Internode Memory Exchange (IMEX) daemon and domain watches
- `x` - ConnectX watches
- `a` - All watches, including IMEX

The independent IMEX watch is useful when an operator needs to distinguish a multi-node fabric-service problem from GPU link health. Enable `e` alongside only the other systems relevant to the workload, or use `a` for the standard all-system check. See [dcgmi health](#) for exact selector, result, and recovery semantics.

To view the current status of all GPUs in the group the client can simply query for the overall group health. The result is an overall health score for the group as well as individual results for each impacted GPU, identifying key problems.

For example, DCGM would show the following when excessive PCIe replay events or InfoROM issues are detected:

```
$ dcgmi health -g 1 -c
```

```
Health Monitor Report
+-----+
| Group 1          | Overall Health: Warning          |
+=====+
| GPU ID: 0        | Warning                          |
|                  | PCIe system: Warning - Detected more than 8 PCIe |
|                  | replays per minute for GPU 0: 13 |
+-----+
| GPU ID: 1        | Warning                          |
|                  | InfoROM system: Warning - A corrupt InfoROM has been |
|                  | detected in GPU 1.                |
+-----+
```

Failure Conditions

System	Error Code	Cause
Any	DCGM_FR_FALLEN_OFF_BUS	A GPU has fallen off the bus.
Any	DCGM_FR_XID_ERROR	One or more “devastating” XIDs associated with a critical hardware error has occurred.
Driver	DCGM_FR_GPU_RECOVERY_DRAIN_P2P	A GPU requires peer-to-peer traffic to be quiesced.
Driver	DCGM_FR_GPU_RECOVERY_DRAIN_RESET	A GPU is operating at reduced capacity due to a fault.
Driver	DCGM_FR_GPU_RECOVERY_REBOOT	A GPU fault may have left the operating system in an inconsistent state.
Driver	DCGM_FR_GPU_RECOVERY_RESET	A GPU requires a reset to recover from a fault.
Driver	DCGM_FR_GPU_RECOVERY_IMEX_DOMAIN	GPU recovery requires restoring the IMEX domain. This driver recovery result is distinct from an IMEX health-watch incident.

System	Error Code	Cause
PCIe	DCGM_FR_PCI_REPLAY_RATE	There were more than 8 PCIe replays in the last minute.
PCIe	DCGM_FR_XID_ERROR	One or more XIDs associated with bus or fabric errors were reported.
Memory	DCGM_FR_FAULTY_MEMORY	Faulty memory elements were detected on a GPU.
Memory	DCGM_FR_VOLATILE_DBE_DETECTED	One or more volatile DBEs are reported (non-recoverable memory errors since the last GPU reset)
Memory	DCGM_FR_PENDING_PAGE_RETIREMENTS	There is one or more pending page retirement.
Memory	DCGM_FR_RETIRED_PAGES_LIMIT	63 or more retired pages have occurred on one GPU.
Memory	DCGM_FR_RETIRED_PAGES_DBE_LIMIT	There have been more than 15 total retired pages due to DBEs and at least one in the last week.
Memory	DCGM_FR_ROW_REMAP_FAILURE	One or more row remapping failures occurred (XID 64 occurred).
Memory	DCGM_FR_UNCONTAINED_ERROR	A GPU experienced an uncontained error (XID 95 occurred).
Memory	DCGM_FR_CONTAINED_ERROR	A GPU reported a contained error (XID 94 occurred).
Memory	DCGM_FR_XID_ERROR	One or more XIDs associated with memory management units (MMU), push buffer direct management access (PBDMA) units, or Reset Channel Verification were reported.
Inforom	DCGM_FR_CORRUPT_INFOROM	Inforom corruption detected (XID 93). DCGM raises this warning only when XID 93 is observed on Volta GPUs; on all other architectures

System	Error Code	Cause
		DCGM ignores XID 93 and does not raise this health warning.
Thermal	DCGM_FR_CLOCKS_EVENT_THERMAL	A thermal violation was detected on a GPU in the last minute.
Thermal	DCGM_FR_FIELD_THRESHOLD_DBL	A CPU's temperature one minute ago and its current temperature are both above the warning temperature.
Thermal	DCGM_FR_FIELD_THRESHOLD_DBL	A CPU's current temperature is above the critical temperature.
Thermal	DCGM_FR_THERMAL_VIOLATIONS	Thermal violations were detected on a GPU.
Thermal	DCGM_FR_XID_ERROR	One or more XIDs associated with EDPP Power Brake Thermal Limit were reported.
Power	DCGM_FR_CLOCKS_EVENT_POWER	A GPU had a power violation detected in the last minute.
Power	DCGM_FR_FIELD_THRESHOLD_DBL	A CPU's power usage is above its power limit.
Power	DCGM_FR_POWER_UNREADABLE	Cannot reliably read the power usage for a GPU.
Power	DCGM_FR_XID_ERROR	One or more XIDs associated with power state change or clock change was reported.
IMEX	DCGM_FR_IMEX_UNHEALTHY	The IMEX daemon or domain reports an unhealthy state.
NVLink	DCGM_FR_FABRIC_PROBE_STATE	A GPU has a non-ready fabric state.
NVLink	DCGM_FR_FIELD_VIOLATION	The fabric health mask reports an unhealthy state.
NVLink	DCGM_FR_NVLINK_ERROR_CRITICAL	An NVLink reported 1 or more errors. This includes datalink CRC,

System	Error Code	Cause
		recovery, and replay errors (pre-Blackwell GPUs) and link recovery events (Blackwell+ GPUs).
NVLink	DCGM_FR_NVLINK_ERROR_THRESHOLD	An NVLink reported a rate of errors which exceeds the minimum threshold.
NVLink	DCGM_FR_NVLINK_CRC_ERROR_THRESHOLD	An NVLink reported more than 100 CRC errors per second.
NVLink	DCGM_FR_NVLINK_EFFECTIVE_BER_THRESHOLD	An NVLink reported effective bit error rate which exceeds the minimum threshold.
NVLink	DCGM_FR_NVLINK_SYMBOL_BER_THRESHOLD	An NVLink reported symbol bit error rate which exceeds the minimum threshold.
NVLink	DCGM_FR_XID_ERROR	One or more XIDs associated with NVLink flow control, chip-to-chip (C2C) link correction, or fabric linear addressing (FLA) privilege were reported.
NVSwitch Fatal	DCGM_FR_NVSWITCH_FATAL_ERROR	An NVSwitch reported one or more fatal errors.
NVSwitch Fatal	DCGM_FR_NVLINK_DOWN	One or more NVLinks is being reported as down.
NVSwitch Non-Fatal	DCGM_FR_NVSWITCH_NON_FATAL_ERROR	An NVSwitch reported one or more non-fatal errors.
ConnectX	DCGM_FR_FIELD_VIOLATION	A ConnectX device reported an unhealthy status.
ConnectX	DCGM_FR_FIELD_VIOLATION	A ConnectX device reported PCIe Advanced Error Reporting (AER) uncorrectable errors (fatal or non-fatal).

Note

The `dcgm` interfaces above only report current health status. The underlying data, exposed via other interfaces, captures more information about the timeframe of the events and their connections to executing processes on the GPU.

Active Diagnostics

Passive health monitoring and active diagnostics answer different questions. Health watches look for



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.